



9

33

3120 sp

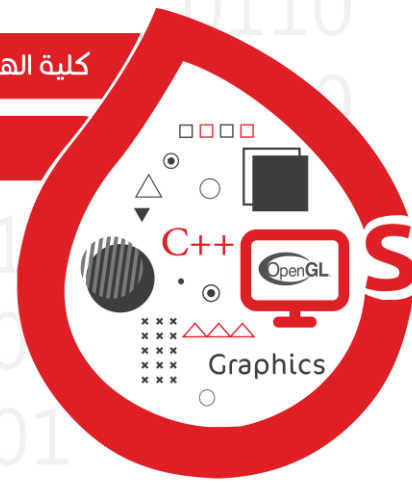
نظري

كلية الهندسة المعلوماتية

السنة الثالثة

Scan Converting 2

د. عمار النحاس



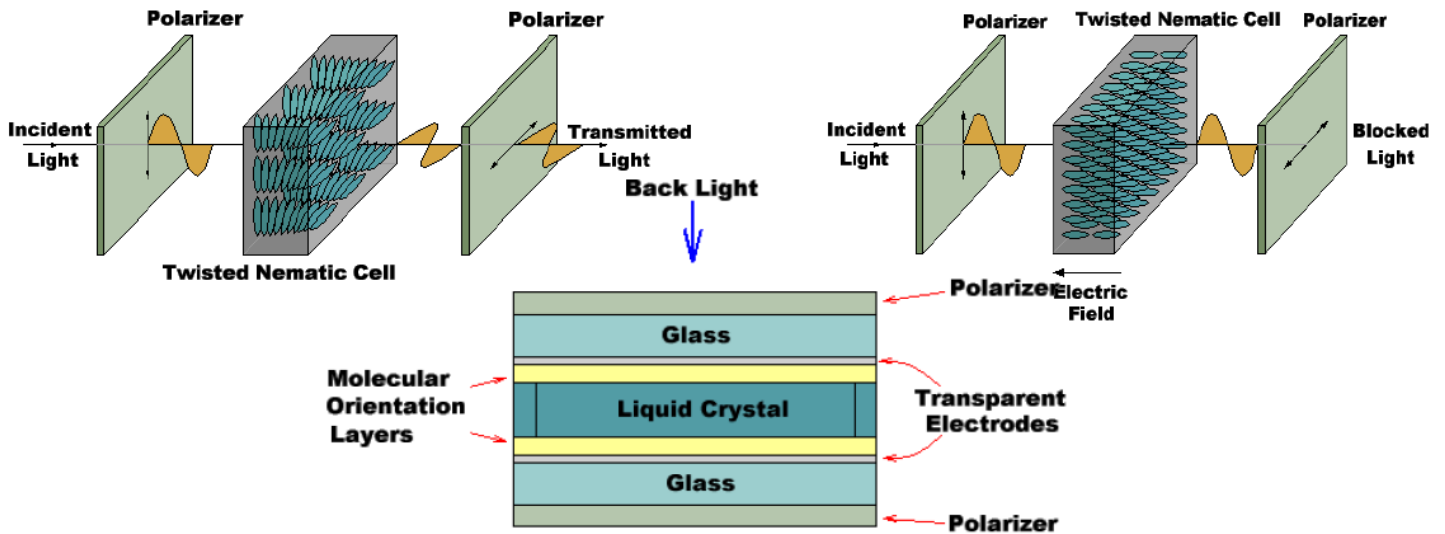
RB Informatics; 2/12/2025

بيانيات حاسوبية

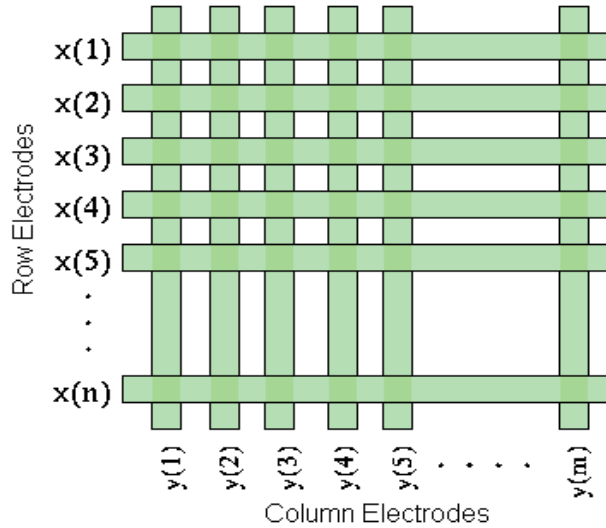
تكلما في المحاضرة السابقة عن أنواع الشاشات وسنكمل في أنواع أخرى:

(LCDs) Liquid Crystal Displays :

- الشاشات البلورية السائلة (LCDs) هي جزيئات عضوية .
- تميل في غياب القوى الخارجية، إلى التبع في تراكيب بلورية. في حالتها غير المثارة أو حالتها البلورية
- تقوم LCDs بدوران استقطاب الضوء بمقدار 90 درجة. عند تطبيق قوة خارجية، تعيد ترتيب نفسها وكأنها سائلة
- في Presence of an electric field، تتجه الشحنات الكهربائية الصغيرة في الجزيئات مع المجال الكهربائي المطبق
- تستخدم LCDs كريستالات سائلة من نوع النيماتيك الملتوية لأنها تتفاعل بشكل يمكن التنبؤ به مع التيار الكهربائي، بحيث تتحكم في مرور الضوء
- حالياً، البديل الأكثر شهرة للشاشات CRT هو شاشات LCD.



- تستخدم شاشات المصفوفة الساكنة شبكة من الأقطاب الكهربائية للتحكم في الفولتية، ولكن المشكلة تكمن في بطئها في التبديل وسهولة حدوث التدفقات الزائدة
- أما شاشات المصفوفة النشطة، فتحتوي على ترانزستور عند كل خلية، وتستخدم بلورة ذات قدرة على التبديل بسرعة أعلى وترانزستورات تحفظ الشحنة وتمنع التدفقات الزائدة.
- وتُستخدم الفلاتر الملونة للحصول على عرض ألوان



مزايا شاشات العرض البلورية السائلة:

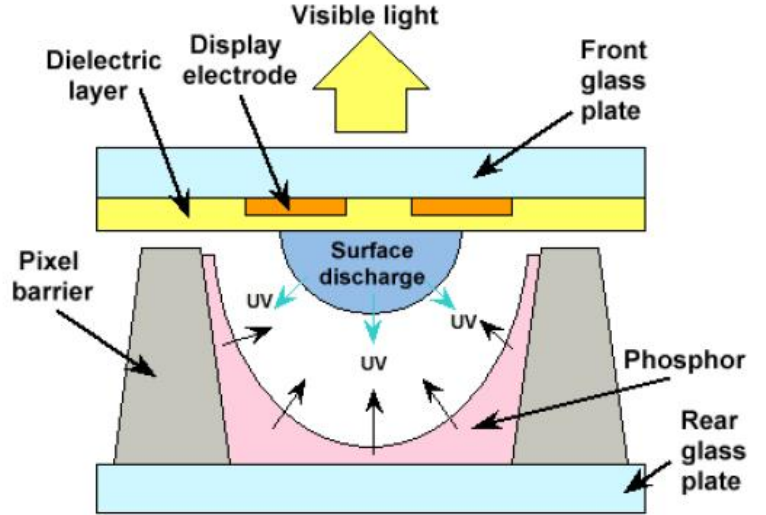
1. خفيفة الوزن
2. استهلاك منخفض للطاقة
3. شاشة مسطحة تماماً، بدون أخطاء هندسية
4. شاشات كبيرة (أكثر من 20 بوصة) على أجهزة الكمبيوتر المكتبية
5. صور واضحة، ألوان رقمية وموحدة
6. لا تصدر إشعاعات كهرومغناطيسية

عيوب شاشات العرض البلورية السائلة:

1. سعر مرتفع (حالياً ثلاثة أضعاف سعر شاشة CRT)
2. زاوية مشاهدة ضعيفة (عادة حوالي 50 درجة + أو -)
3. تباين وسطوع منخفضين
4. عملية الانتقال من الحالة البلورية إلى الحالة السائلة بطيئة للغاية

: (PDPs) Plasma Display Panels

هي في الأساس عبارة عن مصفوفة من الأنابيب الفلورية الصغيرة جدًا

مزايا شاشات البلازما:

1. واحدة للشاشات ذات التنسيقات الكبيرة.
2. هي في الأساس أنابيب فلورية
3. يُشير تفريغ الجهد العالي خليطاً غازياً (هيليوم، زينون).
4. عند الاسترخاء، ينبعث ضوء فوق بنفسجي.
5. يُثير الضوء فوق البنفسجي الفوسفور.
6. زاوية رؤية كبيرة
7. أقل كفاءة من شاشات CRTs

عيوب شاشات البلازما:

1. ليست ساطعة بنفس القدر .
2. تستهلك طاقة أكبر .
3. بكسلات كبيرة (حوالي 1 مم مقارنة بـ 0.2 مم لشاشة CRTs)
4. قصف الأيونات يستنفد الفوسفور .

Light Emitting Diodes (LED display):

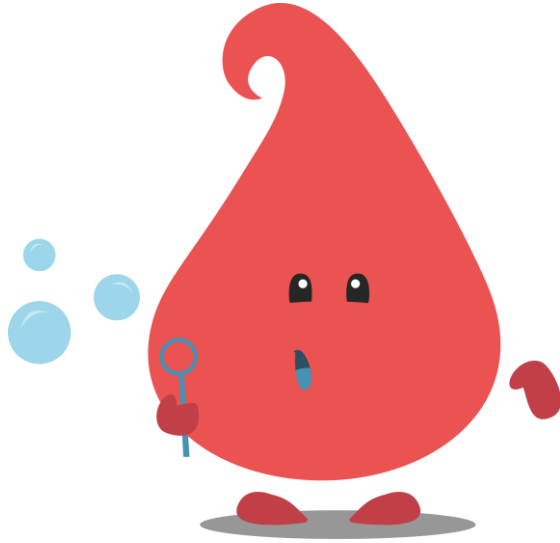
تم اكتشاف وجود الديود و الذي يطلق ضوء، و بالتالي يتم وضع 3 ديودات في كل pixel لعرض الألوان، أي أن الشاشة عبارة عن بكسلات كل بكسل فيه ثلاث ديودات.

محاسنها:

■ نرى الصورة من أي زاوية.

مساوئها:

■ أحياناً تنضرب الآلات، ليس لها حل إلا تغيير الشاشة.



انتبهنا من الكلام عن أنواع الشاشات وسنبداً بـ Scan Converting

يتعلق هذا الموضوع بكيفية تحويل الأشكال الهندسية الأساسية (كالخطوط والدوائر والمضلعات) من تمثيلها الرياضي المجرد (التمثيل الهندسي) إلى مجموعة من البكسلات المضاءة على شاشة الحاسوب (التمثيل النقطي)

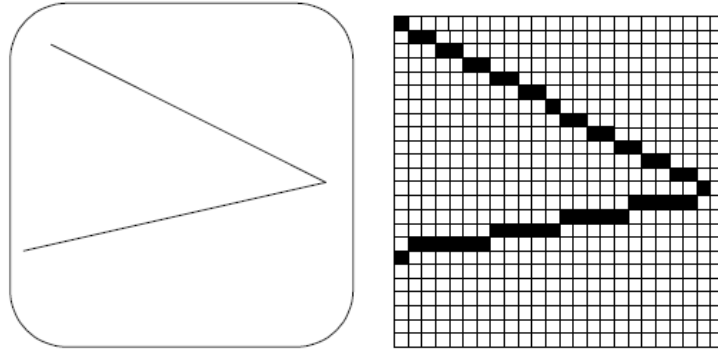
Vector device & Raster device

:vector device

يولد الخط بتحريك حزمة إلكترونية بسلاسة من نقطة البداية إلى نقطة النهاية (مثل شاشات الرسوم المتجهة القديمة) فينتج خطوط ملساء وناعمة.

:raster device

تم استخدام تحويل المسح (Scan Conversion) أو التحويل الراسخري (Rasterization). يتم تحويل الخط من تمثيل هندسي إلى تمثيل نقطي/راسخري، أي توليده كسلسلة من النقاط المنفصلة (البكسلات). فينتج خطوط ذات مظهر متعرج لأن الخط لا يمكن أن يمر إلا عبر مراكز البكسلات.



صفات الخطوط:

عندما نقوم بتحويل المسح لخط ما، يجب أن تضمن الخوارزمية إظهار الناتج بالصفات التالية لجعل الخط يبدو طبيعياً ومقبولاً للمستخدم

- إنهاء مناسب: يجب أن يبدأ الخط وينتهي بدقة عند البكسلات المقابلة لنقطتي البداية والنهاية الهندسيتين
- مظهر متسم: يجب ألا تظهر فجوات في الخط
- كثافة ثابتة تقريباً: يجب ألا يكون الخط سميكاً جداً في جزء ورقيقاً جداً في جزء آخر
- سماكة موحدة: يجب الحفاظ على سماكة الخط (عرضه) بشكل ثابت على طوله

كفاءة الحوسبة:

لتحقيق أقصى سرعة، خاصة وأن الشاشات الحديثة تتطلب معالجة ملايين البكسلات، يجب أن تعتمد الخوارزميات على:

- استخدام الجمع بدلاً من الضرب
- التعامل مع الأعداد integer بدلاً من float
- تشغيل البكسلات الأقرب إلى الخط المثالي

Scan converting primitives

:Scan converting

تم تطوير خوارزميات محددة لتحقيق:

■ رسم الخط (line drawing):

خوارزمية بريزيناها (Bresenham, 1965)

■ الدوائر:

خوارزمية بريزيناها (Bresenham, 1977)

■ توسيعات (Extensions):

يمكن تطبيق نفس المبادئ والتقنيات على أشكال أكثر تعقيداً مثل القطع الناقص والقطع المكافئ والقطع الزائد

ملاحظة هامة: هذه الخوارزميات مطبقة حالياً في جميع مكتبات الرسومات (OpenGL, DirectX, Vulkan)

تطبيقات هذه الخوارزميات لمجالات مشابهة: حركة الروبوت - تصوير الحجم

تحسينات المستوى المنخفض:

هي تقنيات لزيادة سرعة الخوارزمية بشكل كبير من خلال الاستفادة من خصائص معينة للمعالج أو نظام التشغيل. على الرغم من أنها قد تكون "محل شك" لأنها تعتمد على تفاصيل الجهاز، إلا أن هناك قواعد عامة متسقة عبر معظم الأجهزة:

- حسابات الأعداد الصحيحة أسرع من الفاصلة العائمة
- الجمع والطرح أسرع من الضرب
- الضرب أسرع من القسمة
- استخدام الجداول أسرع من الحوسبة: بدلاً من إعادة حساب قيمة ما مراراً وتكراراً، يتم تخزينها في جدول والوصول إليها مباشرة
- تجنب الحوسبة غير الضرورية: أي لتجنب إجراء العمليات الحسابية المعقدة عند عدم الحاجة إليها.

والآن سنتحدث عن خوارزميات رسم مستقيم ودائرة في raster display:

■ Scan Converting lines:

تكمن المشكلة الأساسية في أن الخط (كمفهوم هندسي) مستمر، بينما الشاشة (الجهاز الراسخري) لا تعرض إلا نقاطاً منفصلة تسمى بكسلات.

تمثل البكسلات عند قيم إحداثية صحيحة (Integer coordinate values).

لأي قطعة مستقيمة بين نقطتين (x_1, y_1) و (x_2, y_2) . يمكن تمثيلها بالمعادلة الصريحة للمستقيم:

$$y = mx + b$$

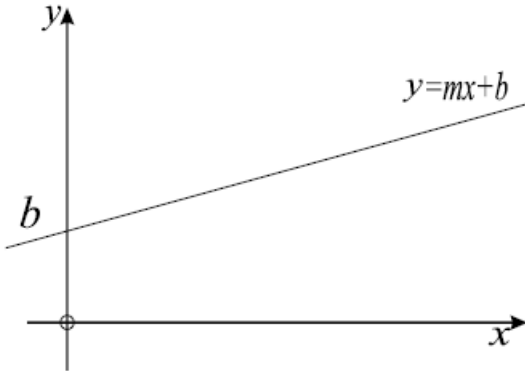
حيث:

▪ m هو ميل الخط (Slope) ويُحسب كالتالي:

$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

▪ b مقدار الإزاحة على المحور y ويُحسب كالتالي:

$$b = y_1 - mx_1$$



▪ فأن أي بكسل يحقق هذه المعادلة $y = mx + b$ فهي موجودة ضمن المستقيم أي ستضيئ و إذا قمنا بتجريب كل البكسلات الموجودة على الشاشة و اختبرناها فهل تحقق المعادلة أو لا فإن حققت أضيئت وإذا لم تحقق لم تضيئ فتحصل على المستقيم المطلوب، لكن هذه الطريقة صعبة ومكلفة جداً ولا نريدها.

▪ نستنتج طريقة خوارزمية scan converting و هذه الطريقة لها شروط:

▪ أن يرسم المستقيم فقط في منتصف الربع الأول أي فقط المستقيمات التي ميلها $0 < m < 1$.

▪ أن يكون للمستقيم عرض ثابت تقريباً و أن تكون الخوارزمية فعالة أي تعمل بأسرع ما يمكن.

لكن يوجد لدينا مشكلة، كيف سنحصل على المستقيمات الأخرى؟

▪ عن طريق التناظر بالنسبة لمحور ال y و بالنسبة لمنتصف الربع الأول $y = x$ إذ إن لكل مستقيم ميلان نتحكم بها و نعيدها إلى تحت منتصف الربع الأول بالتناظر، و بالتالي نوجد النقاط ثم نأخذ نظيراتها مجدداً.

▪ يمكن أيضاً تمثيل قطعة الخط بالصيغة الباراميتريّة (Parametric form):

$$x = x_1 + (x_2 - x_1)t$$

$$y = y_1 + (y_2 - y_1)t$$

حيث: $0 \leq t \leq 1$.

التعامل مع الميل (Slope Handling):

ميل الخط m يمكن أن يكون أقل من 1 أو أكبر من 1، ويمكن أن يكون موجباً أو سالباً.

تصمم هذه الخوارزميات (مثل بريزنهايم) في الأساس لحالات الميل المحصورة بين 0 و 1 ($0 \leq m \leq 1$).

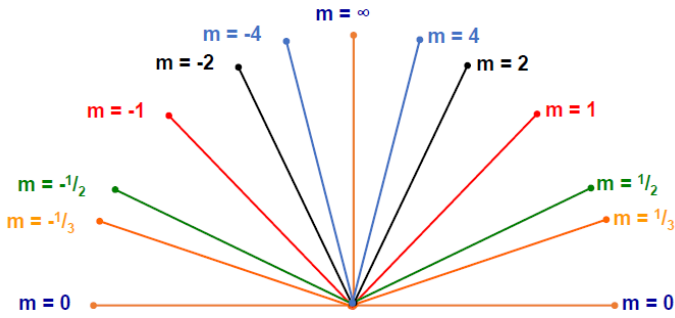
إذا كان الميل أكبر من 1 ($m > 1$) أو كان خطاً

عمودياً: نقوم بتبديل المحورين x و y (Swap y and x)، ونرسم الخط، ثم نعكس التبديل في

الإخراج.

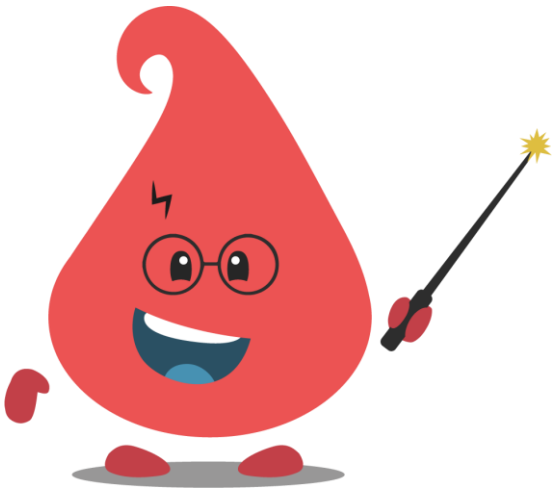
إذا كان الميل سالباً: نرسم الخط كما لو كان موجباً

ثم نعكسه (Flip it).



ملاحظة ارييسيزية:

إذا بدلنا بين نقطة البداية و نقطة النهاية في المستقيمات لا يؤثر ولكن اصطلاحاً ال y تبعها أقل هي نقطة البداية.



1. (Naïve Algorithm):

بما اننا نعلم أن قيمة x ستزداد بمقدار 1 بين كل بكسل، و البكسل الذي يليه. وبما اننا نستطيع إيجاد معادلة المستقيم باستخدام نقطة البداية والنهاية. بالتالي يمكننا حساب قيمة y لكل نقطة بتعويض x بالمعادلة.

تعتمد هذه الخوارزمية على تطبيق مباشر لمعادلة الخط:

$$y = mx + b$$

- $y_0 = mx_0 + b$
- $y_1 = mx_1 + b$
- $m = \frac{y_1 - y_0}{x_1 - x_0} = \frac{dy}{dx}$
- $b = y_0 - mx_0$

```
Void Line (
    int x0, int y0,
    int x1, int y1)
{
    int dx = x1 - x0;
    int dy = y1 - y0;
    WritePixel(x0, y0);
    if (dx != 0)
    {
        float m = (float) dy / (float) dx;
        float b = y0 - m * x0;
        dx = (x1 > x0) ? 1 : -1;
        while (x0 != x1)
        {
            x0 += dx;
            y0 = Round(m * x0 + b);
            WritePixel(x0, y0);
        }
    }
}
```

مثال على هذه الخوارزمية:

سؤال هام:

مستقيم بدايته (2,3) و نهايته (15,10)

$$15 - 2 + 1 = 14 \text{ pixels}$$

$$y - y_0 = m(x - x_0) = y - 3 = \frac{7}{13}(x - 2)$$

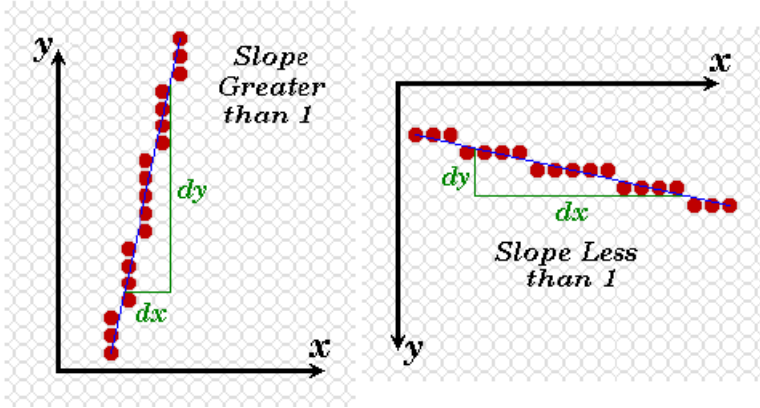
$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

$$m = \frac{10 - 3}{15 - 2} = \frac{7}{13}$$

x_i	y_i	y
2	3	3
3	$3 + \frac{7}{13}$	3
4	$3 + \frac{14}{13}$	4
5	$3 + \frac{21}{13}$	4
6	$3 + \frac{28}{13}$	5

و نتابع لحد ال 15 = x

Rotate and Rename coordinate axes

مشاكل هذه الخوارزمية:

- غير فعالة تتطلب حسابات مكلفة لكل بكسل
- عمليتان على الاصلة العائمة لكل بكسل
- عملية ضرب واحدة على الفاصلة العائمة
- عملية جمع واحدة على الفاصلة العائمة
- استدعاء لدالة التقريب

2. (Basic Incremental Algorithm - DDA):

بما أن ميل المستقيم ثابت اذاً نعلم أنه كلما زادت x بمقدار 1 ستزداد قيمة y بمقدار ثابت m الميل .

تُعرف هذه الخوارزمية باسم محلل التفاضل الرقمي (Digital

Differential Analyzer - DDA)

هدفها هو تجنب عمليات الضرب في كل خطوة.

مبدأ العمل:

بدلاً من إعادة حساب y_{i+1} بالكامل باستخدام الضرب (كما في الخوارزمية السابقة)، نستخدم علاقة الزيادة:

$$1. \text{ نعلم أن } y_{i+1} = mx_{i+1} + b$$

$$2. \text{ بما أننا نزيد } x \text{ بمقدار } \Delta x \text{ (وغالباً } \Delta x = 1 \text{)}: x_{i+1} = x_i + \Delta x$$

$$3. \text{ بالتعويض: } y_{i+1} = m(x_i + \Delta x) + b = (mx_i + b) + m\Delta x$$

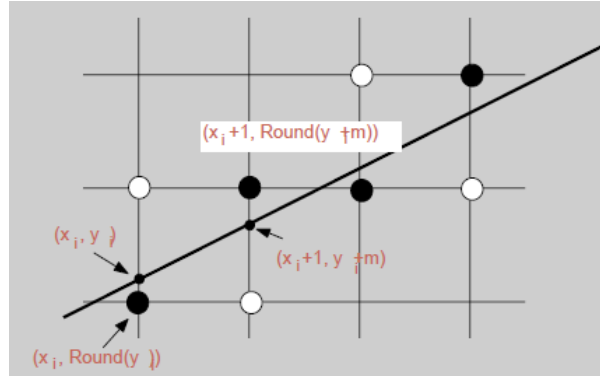
$$4. \text{ وبما أن } (mx_i + b) = y_i, \text{ فإن العلاقة التزايدية تصبح: } y_{i+1} = y_i + m\Delta x$$

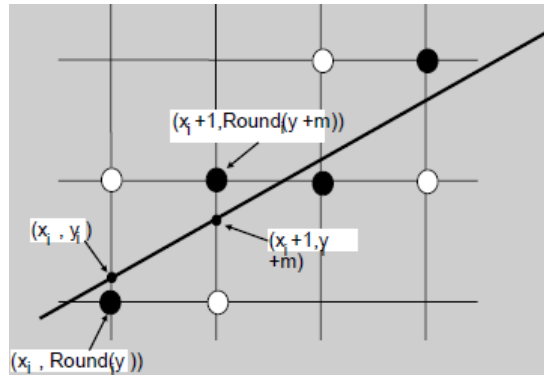
$$5. \text{ إذا كانت } \Delta x = 1 \text{ (وهي الحالة الشائعة عندما يكون الميل } m \leq 1 \text{)} \text{ فإننا نحصل على:}$$

$$x_{i+1} = x_i + 1$$

$$y_{i+1} = y_i + m$$

- ميزة: تم التخلص من عملية الضرب واستبدالها بعملية جمع بسيطة (إضافة الميل m) في كل خطوة.
- عيوب متبقية: لا تزال تستخدم الفاصلة العائمة (m) وتتطلب التقريب (Rounding) لتحديد البكسل الصحيح.





```

Void Line (                /* Assumes -1 <= m <= 1, x0 < x1 */
    int x0, int y0,        /* Left end point */
    int x1, int y1,        /* Right end point */
    int value)             /* Value to place in line's pixels */
{
    int x;                 /* x runs from x0 to x1 in unit increments */

    Double dy = y1 - y0;
    Double dx = x1 - x0;
    Double m = dy / dx;

    Double y = y0;

    for (x = x0; x <= x1; x++)
    {
        WritePixel(x, Round(y), value); /* Set pixel to value */
        y += m;                         /* Step y by slope m */
    }
} /* Line */

```



تحسين خوارزمية DDA (الإصدار الثاني v2): تجنب التقريب

الهدف التالي هو تجنب عملية التقريب لأنها مكلفة وبالتالي نتجنب أن تأخذ العملية وقتاً طويلاً ونتجنب أن نرسم النقطة نفسها مرتين

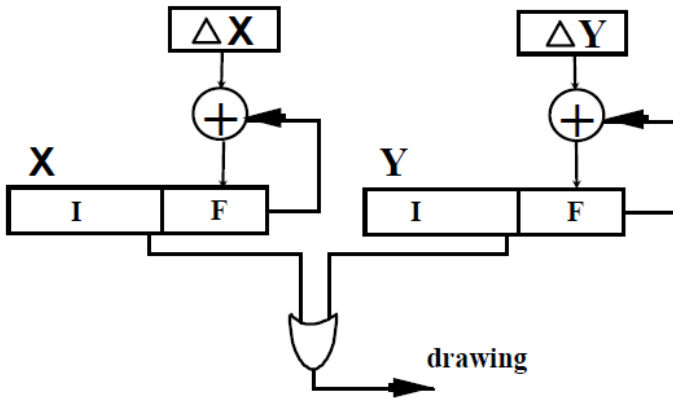
ويتم ذلك باستخدام مفهوم الفائض الحسابي (arithmetic overflow):

■ يتم الاحتفاظ بقيمة x و y في المسجلات (Registers) لها جزءان:

- صحيح (Integer).
- كسري (Fractional).

■ تتم إضافة القيمة التزايدية بشكل متكرر إلى الجزء الكسري.

■ عندما يفيض الجزء الكسري (يتجاوز 1)، تتم زيادة الجزء الصحيح المقابل تلقائياً، وهذا هو ما يمثل عملية اختيار البكسل التالي دون الحاجة إلى دالة تقريب صريحة.



مثال امتحاني:

مستقيم له نقطة بداية (3,2) و نقطة النهاية (11,6)

$$m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{4}{8} = \frac{1}{2}$$

$$b = y_1 - mx = 2 - \left(\frac{1}{2}\right)(3) =$$

$$y = \frac{1}{2}x + \frac{1}{2}$$

x	$y \rightarrow y = \frac{1}{2}x + \frac{1}{2}$
3	نقطة البداية: 2
4	$2.5 \approx 3$
5	3
6	$3.5 \approx 4$
7	4
8	$4.5 \approx 5$
9	5
10	$5.5 \approx 6$
11	نقطة النهاية: 6

- حصلنا على نقطة البداية و النهاية المطلوبة، لكن من شروطنا أن تكون الخوارزمية سريعة و هذه الخوارزمية بطيئة جداً ، لماذا؟
- لأن كل تعويض له عملية جمع و ضرب و تقريب أيضاً و لتسريعها نستخدم float إذا كان int ، حيث سيكون أسرع، أي أننا مضطرين لتشغيل بكسلات لا تقع تماماً على المستقيم و إنما مقارنة و هذا ما يفسر تكسر المستقيم في الرسم العملي

نعيد المثال السابق :

$m = 1 / 2$	
x	$y = y + m$
3	أول Y:2
4	$2.5 \approx 3$
5	3
6	$3.5 \approx 4$
7	4
8	$4.5 \approx 5$
9	5
10	$5.5 \approx 6$
11	نقطة النهاية: 6

هام سؤال امتحاني

3. خوارزمية نقطة المنتصف (MidPoint Line Algorithm - Bresenham):

سؤال دورات:

■ هذه الخوارزمية مع أي من الأشكال تعمل؟

تعمل مع جميع الأشكال التي تملك معادلات، ليس فقط مع المستقيمات بل تعمل مع (المستقيمات - الدوائر - الأشكال البيضاوية - قطع ناقص...)

■ لماذا لا نستخدم $y = mx + b$ (Explicit Function) للخط؟

في البداية، فكرنا باستخدام المعادلة الصريحة

المشكلة: هذه الطريقة تتطلب عمليات ضرب وقسمة وأرقاماً عائمة وهو أمر مكلف جداً من الناحية الحسابيةالحل: خوارزمية بريزينهام تعتمد فقط على الأعداد الصحيحة

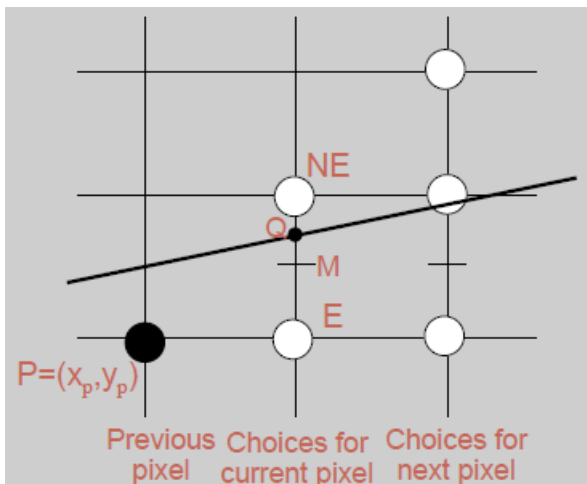
(Integers) وعمليات الجمع والطرح، مما يجعلها سريعة جداً

عند تنفيذها على مستوى العتاد (Hardware)، فنستخدم الدالة

الضمنية للخط:

$$F(x, y) = ax + by + c = 0$$

$$\text{if } dx = x_1 - x_0 \text{ and } dy = y_1 - y_0$$



$$y = \frac{dy}{dx} \cdot x + B$$

$$F(x, y) = dy \cdot x - dx \cdot y + B \cdot dx = 0$$

$$a = dy,$$

$$b = -dx,$$

$$c = B$$

هذه الدالة لها خاصية سحرية:

- إذا كان $F(x, y) = 0$: النقطة تقع على الخط تماماً.
- إذا كان $F(x, y) > 0$: النقطة تقع أسفل الخط.
- إذا كان $F(x, y) < 0$: النقطة تقع أعلى الخط.

نقطة المنتصف (M):

آلية اتخاذ القرار: $P = (x_p, y_p)$

في الخطوة التالية، نزيد الإحداثي x بمقدار 1

تخيل أننا قمنا للتو برسم البكسل الحالي:

دائماً (بفرض أن الميل بين 0 و 1)

الآن، أمامنا خياران للبكسل القادم:

1. البكسل الشرقي (E): البقاء على نفس الارتفاع $(x_p + 1, y_p)$

2. البكسل الشمالي الشرقي (NE): الصعود درجة للأعلى $(x_p + 1, y_p + 1)$

كيف نختار؟

نضع نقطة اعتبار تسمى **نقطة المنتصف (Midpoint-M)** وهي تقع بالضبط بين E و NE عند الإحداثيات:

$$(x_p + 1, y_p + 0.5)$$

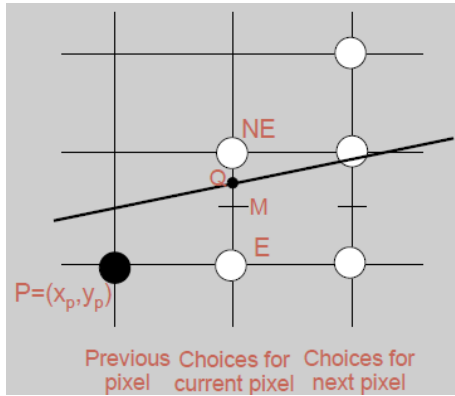
للتوضيح:

نلاحظ أن البكسل الذي نريد اضاءته سيكون دوماً إما البكسل المجاور $(x+1, y)$ أو البكسل الذي فوقه $(x+1, y+1)$

قاعدة الاختيار (The Decision Rule):

نقوم بتعويض إحداثيات نقطة المنتصف M في الدالة $F(x, y)$:

1. الحالة الأولى: إذا كانت $F(M) < 0$:



- هذا يعني أن نقطة المنتصف تقع أعلى الخط.
- بالتالي، الخط الحقيقي أقرب إلى البكسل E (الشرقي).
- القرار: نختار البكسل E.

2. الحالة الثانية: إذا كانت $F(M) > 0$:

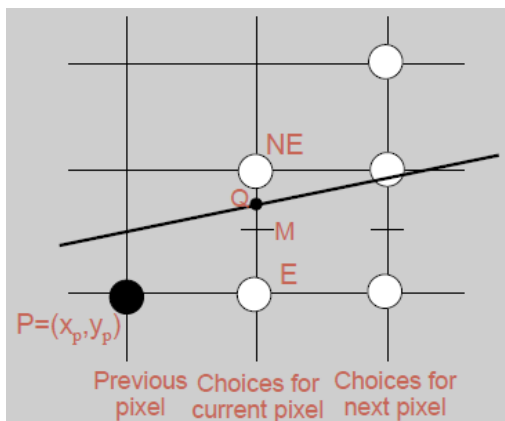
- هذا يعني أن نقطة المنتصف تقع أسفل الخط.
 - بالتالي، الخط الحقيقي أقرب إلى البكسل NE (الشمال الشرقي).
 - القرار: نختار البكسل NE.
- أقصى خطأ ممكن في هذه الخوارزمية لا يتجاوز نصف بكسل ($\leq \frac{1}{2}$).
إذاً الآن كيف نحسب $F(M)$ ؟

$$F(M) = F\left(x_p + 1, y_p + \frac{1}{2}\right)$$

decision variable d :

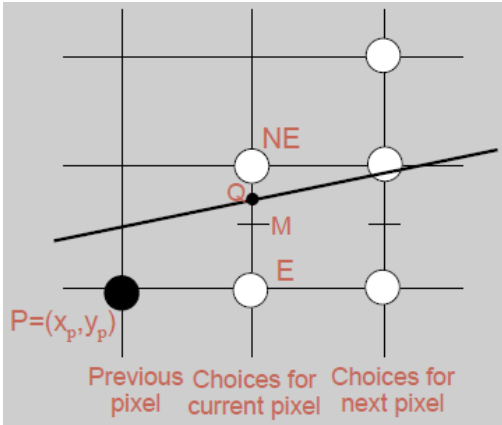
$$d = F\left(x_p + 1, y_p + \frac{1}{2}\right) = a(x_p + 1) + b\left(y_p + \frac{1}{2}\right) + c$$

- If $d > 0$: we choose pixel NE
- If $d < 0$: we choose pixel E



$F(x, y) = 0$ on the line
 $F(x, y) > 0$ below
 $F(x, y) < 0$ above

■ ماذا يحدث لقيمة d إذا اخترنا النقطة E؟



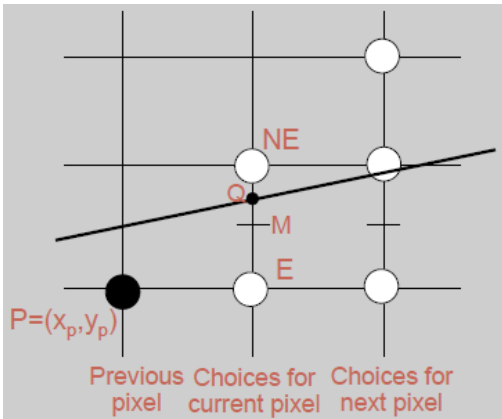
نتحرك أفقياً فقط فتكون:

$$d_{new} = F\left(x_p + 2, y_p + \frac{1}{2}\right) = a(x_p + 2) + b\left(y_p + \frac{1}{2}\right) + c$$

$$d_{old} = F\left(x_p + 1, y_p + \frac{1}{2}\right) = a(x_p + 1) + b\left(y_p + \frac{1}{2}\right) + c$$

$$\Delta E = d_{new} - d_{old} = a \quad \text{with: } a = dy$$

■ إذا اخترنا النقطة NE ماذا يحدث لقيمة NE؟



نتحرك خطوة في x وخطوة في y :

$$d_{new} = F\left(x_p + 2, y_p + \frac{3}{2}\right) = a(x_p + 2) + b\left(y_p + \frac{3}{2}\right) + c$$

$$d_{old} = F\left(x_p + 1, y_p + \frac{1}{2}\right) = a(x_p + 1) + b\left(y_p + \frac{1}{2}\right) + c$$

$$\Delta E = d_{new} - d_{old} = a + b \quad \text{with: } a = dy - dx$$

We can derive the value of the decision variable adding

$$\Delta E = dy - dx$$

■ تعلمنا كيف نحدث قيمة d أثناء السير على الخط لكن من أين نبدأ أصلاً؟ وما هي أول قيمة لـ d ؟

$$F(M) = d$$

حيث:

$$c + by + ax = F(x, y) \quad \blacksquare$$

$$M = \text{نقطة المنتصف بين E و NE} \quad \blacksquare$$

أول بكسل هو: (x_0, y_0)

أول نقطة منتصف نختبرها: $(x_0 + 1, y_0 + \frac{1}{2})$

$$\text{and } F(x_0 + 1, y_0 + \frac{1}{2}) = a(x_0 + 1) + b(y_0 + \frac{1}{2}) + c$$

$$= a x_0 + b y_0 + c + a + \frac{b}{2}$$

$$= F(x_0, y_0) + a + \frac{b}{2}$$

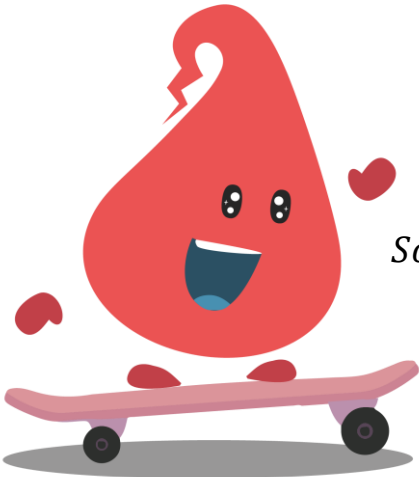
$$\text{with } F(x_0, y_0) = 0$$

$$\text{So } d_{\text{initial}} = a + \frac{b}{2} = dy - \frac{dx}{2}$$

$$a = dy,$$

$$b = -dx,$$

$$c = B$$



🔗 مثال عما سبق:

المسألة:

باستخدام خوارزمية نقطة المنتصف، حدد البكسلات اللازمة لرسم الخط الواصل بين نقطة البداية $P_1(1, 1)$ ونقطة النهاية $P_2(8, 5)$:

الحل:

▪ نقاط البداية والنهاية: $(x_1, y_1) = (1, 1)$ و $(x_2, y_2) = (8, 5)$

▪ حساب الفروقات:

$$dx = 8 - 1 = 7 \quad \bullet$$

$$dy = 5 - 1 = 4 \quad \bullet$$

▪ ثوابت التحديث (لتحقيق السرعة وتجنب الضرب داخل الحلقة):

$$2dy = 2 \times 4 = 8 \quad \bullet$$

$$2dy - 2dx = 8 - 14 = -6 \quad \bullet$$

▪ قيمة متغير القرار الابتدائي (p_0) :

$$p_0 = 2dy - dx = 8 - 7 = 1 \quad \bullet$$

قواعد اتخاذ القرار (Algorithm Logic):

في كل خطوة k ، نزيد x بمقدار 1 دائماً، ونقرر قيمة y :

1. إذا كان $p_k \geq 0$: نختار البكسل NE (شمال شرق)، أي نزيد y بمقدار 1.

$$p_{k+1} = p_k + (2dy - 2dx)$$

2. إذا كان $p_k < 0$: نختار البكسل E (شرق)، أي تبقى y كما هي.

$$p_{k+1} = p_k + 2dy$$

الخطوة (k)	قيمة القرار الحالية (p_k)	القرار المتخذ	الإحداثيات النتيجة (x,y)	قيمة p التالية (p_{k+1})
البداية	---	---	(1, 1)	$p_0 = 1$
1	(موجب) 1	NE	(2, 2)	$1 + (-6) = -5$
2	(سالب) -5	E	(3, 2)	$-5 + 8 = 3$
3	(موجب) 3	NE	(4, 3)	$3 + (-6) = -3$
4	(سالب) -3	E	(5, 3)	$-3 + 8 = 5$
5	(موجب) 5	NE	(6, 4)	$5 + (-6) = -1$
6	(سالب) -1	E	(7, 4)	$-1 + 8 = 7$
7	(موجب) 7	NE	(8, 5)	---

كود هذه الخوارزمية:

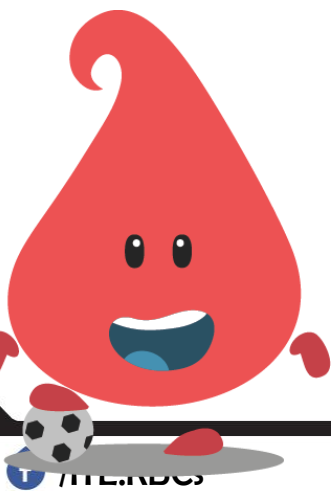
```

Void MidPointLine (
    int x0, int y0,      /* Left end point */
    int x1, int y1,      /* Right end point */
    int value)           /* Value to place in line's pixels */
{
    int x;               /* x runs from x0 to x1 in unit increments */
    int dy = y1 - y0;
    int dx = x1 - x0;
    int d = 2*dy - dx;    /* Initial value of d */
    int incrE = 2*dy;     /* Increment used for move to E */
    int incrNE = 2*(dy - dx); /* Increment used for move to NE */
    int x = x0;
    int y = y0;
    WritePixel(x, y, value); /* The start pixel */
    While (x < x1) {
        if (d <= 0) {      /* Choose E */
            d += incrE;
            x++;
        }
        else {             /* Choose NE */
            d += incrNE;
            x++;
            y++;
        }
        WritePixel(x, y, value); /* The selected pixel closest to the line */
    } /* While */
} /* MidPointLine */

```

مزايا هذه الخوارزمية:

- التعامل مع الأعداد الصحيحة فقط
- استخدام التناظر لتقليل التعقيد
- الاختيار محصور بين بكسلين فقط
- لا تحتاج إلى عمليات الضرب والقسمة
- بسيطة للغاية



- مناسبة للتنفيذ على الأجهزة
- مناسبة للبرمجة بلغة التجميع

🔗 بالنهاية:

تحويل الخطوط إلى صيغة نقطية أمر مهم (شائع جداً) يمكن استخدام هذه التقنية نفسها لتحويل أي دالة متعددة الحدود تقريباً إلى صيغة نقطية (مع بعض التعديلات)

■ Scan Converting Circles:

كيف نرسم دائرة على شاشة Raster؟
الدائرة رياضياً شكل مستمر، لكن الشاشة: شبكة بكسلات يجب أن نقرر أي بكسلات تضيء لتقريب الدائرة
نفس مشكلة الخط، لكن أصعب لأن:

- الميل يتغير في كل نقطة
- الشكل مغلق

1. Naïve Algorithm (V1):

الفكرة:

نفرض مركز الدائرة في الأصل:

$$R^2 = x^2 + y^2$$

نحلها بشكل صريح:

$$y = \pm \sqrt{R^2 - x^2}$$



ثم:

- نمشي x من 0 إلى R.
- نحسب y لكل x.
- نرسم النقطة.

لماذا هذه الطريقة سيئة؟

1. عمليات مكلفة:

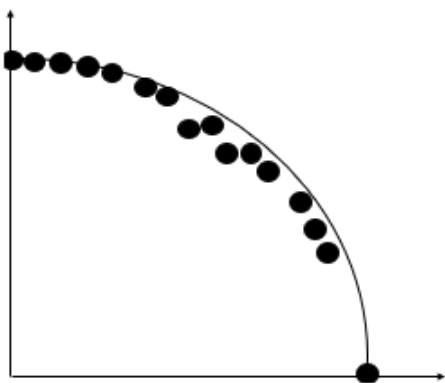
- ضرب
- جذر تربيعي (sqrt) ← بطيء جداً.

2. فجوات كبيرة (Gaps).

عندما $R \approx x$:

- التغير في y كبير.
- تظهر نقاط متباعدة (كما في الرسم).

3. حل جزئي:



- ترسم فقط ربع دائرة.
 - تحتاج التكرار.
- لذلك: غير عملية للهاردوير.

2. Naïve Algorithm (V2) – Eight-Way Symmetry

الدائرة متمثلة حول:

- المحور x
- المحور y
- الأقطار

أي:

- لو لديك نقطة وحدة (x, y)
- هناك 7 نقاط ثمانية على الدائرة مباشرة

التماثل الثماني (8-way symmetry):

من نقطة واحدة (x, y) نولد:

$(x, -y), (y, -x), (-y, -x), (-x, -y), (x, y), (y, x), (x, x), (y, y)$

```

Void CirclePoints (int x, int y , value)
{
    WritePixel (x, y, value);
    WritePixel (y, x, value);
    WritePixel (y, -x, value);
    WritePixel (x, -y, value);
    WritePixel (-x, -y, value);
    WritePixel (-y, -x, value);
    WritePixel (-y, x, value);
    WritePixel (-x, y, value);
} /* CirclePoints */

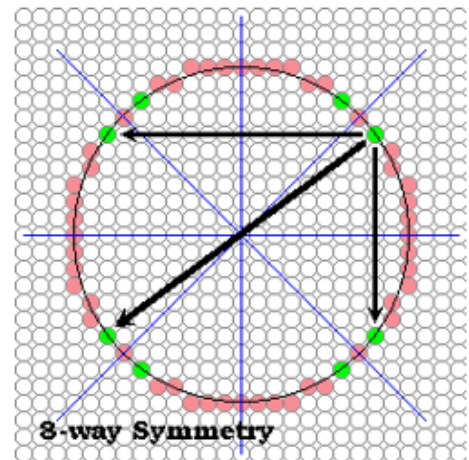
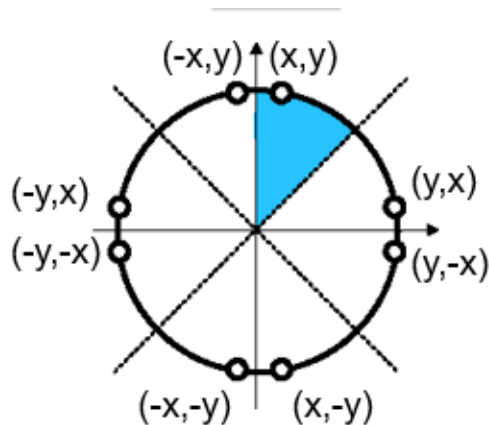
```

الفائدة:

- نرسم ثمن دائرة فقط.
- نقل عدد الحسابات.
- نقل عدد الحالات (slopes).

لكن:

- ما زلنا نستخدم sqrt.
 - ما زلنا نستخدم ضرب.
- أي: تحسين، لكنه ليس الحل المثالي



3. Midpoint Circle Algorithm (Bresenham)

الفكرة الأساسية (نفس منطق الخط):

- بدلاً من: حساب y بالجذر.
- نقوم بـ: الاختيار بين بكسلين محتملين باستخدام نقطة المنتصف.

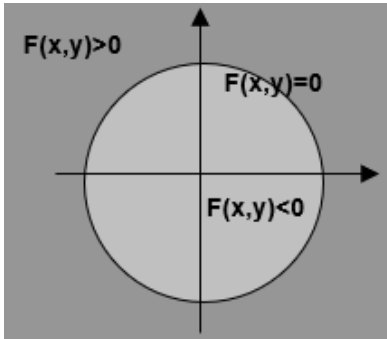
المعادلة الضمنية للدائرة:

بدل المعادلة الصريحة:

$$y = \pm \sqrt{R^2 - x^2}$$

نستخدم المعادلة الضمنية:

$$F(x, y) = x^2 + y^2 - R^2$$



- النقطة على الدائرة: $F(x, y) = 0$

- النقطة داخل الدائرة: $F(x, y) < 0$

- النقطة خارج الدائرة: $F(x, y) > 0$

عند كل خطوة لدينا نقطتان مرشحتان للرسم:

- النقطة الشرقية (E): $(x + 1, y)$
- النقطة الجنوبية الشرقية (SE): $(x + 1, y - 1)$

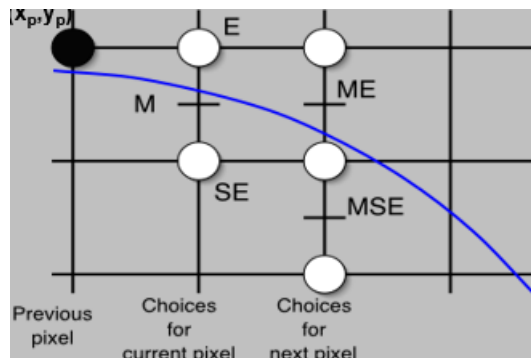
نحسب قيمة الدالة عند نقطة المنتصف:

$$d = F\left(x_p + 1, y_p - \frac{1}{2}\right)$$

وبالتعويض:

$$d = (x_p + 1)^2 + \left(y_p - \frac{1}{2}\right)^2 - R^2$$

هذه القيمة تسمى قيمة القرار لأنها تحدد أي نقطة نختار.



تحديث قيمة القرار (عند اختيار E):

إذا اخترنا النقطة E، فالإحداثيات الجديدة تصبح:

$$y_{new} = y_p, \quad x_{new} = x_p + 1$$

$$d_{new} = F\left(x_p + 2, y_p - \frac{1}{2}\right) = (x_p + 2)^2 + \left(y_p - \frac{1}{2}\right)^2 - R^2$$

$$d_{old} = d_{old} + (2x_p + 3)$$

$$\Delta E = (2x_p + 3)$$

تحديث قيمة القرار (عند اختيار SE):

إذا اخترنا النقطة SE، فالإحداثيات الجديدة تصبح:

$$y_{new} = y_p - 1, \quad x_{new} = x_p + 1$$

$$d_{new} = F\left(x_p + 2, y_p - \frac{3}{2}\right) = (x_p + 2)^2 + \left(y_p - \frac{3}{2}\right)^2 - R^2$$

$$\Delta E = (2x_p - 2y_p + 5)$$

$$\Delta SE = 2(x_p - y_p) + 5$$

■ تعلمنا كيف نحدث قيمة d أثناء السير لكن من أين نبدأ أصلاً؟ وما هي أول قيمة لـ d ؟

في خوارزمية Midpoint Circle نبدأ دائماً من أعلى نقطة في الربع الأول من الدائرة.

بما أن مركز الدائرة هو: $(0,0)$

ونصف القطر هو: R

إذاً أول نقطة رسم هي: $(x_0, y_0) = (0, R)$

هذه النقطة مضمنة على أنها دائرة

النقطة الثانية المحتملة:

بعد رسم النقطة $(R, 0)$ ، لدينا خياران:

■ $E: (1, R)$

■ $SE: (1, R - 1)$

لكن بدلاً من أن نقرر مباشرة، نستخدم نقطة المنتصف بينهما.

نقطة المنتصف الأولى:

نقطة المنتصف بين $(1, R)$ و $(1, R - 1)$ هي: $M = (1, R - \frac{1}{2})$



حساب قيمة القرار الابتدائية:

نستخدم دالة الدائرة:

$$F(x, y) = x^2 + y^2 - R^2$$

ونعوض نقطة المنتصف:

$$d_{initial} = F(1, R - \frac{1}{2})$$

بعد التعويض:

$$d = \frac{5}{4} - R$$

إذاً:

$$\frac{5}{4} < R \Rightarrow 0 > d$$

نقطة المنتصف داخل الدائرة

نختار النقطة E

وبما أن أي دائرة حقيقية نصف قطرها أكبر من 1، فأول خطوة دائماً تقريباً تكون E

ملاحظة: فرق d_{new} بين خوارزمية نقطة المنتصف الخاصة بالمستقيمت وخوارزمية نقطة المنتصف الخاصة بالدوائر في المستقيمت: ثابت يتعلق بـ dx و dy في الدوائر: متغير لأنه متعلق بآخر نقطة.

كود هذه الخوارزمية:

```

Void MidPointCircle ( /* Assumes center of circle is at origin */
    int radius,
    int value) /* Value to place in line's pixels */
{
    int x = 0;
    int y = radius;
    double d = 5.0/4.0 - radius; /* Initial value of d */
    CirclePoints(x, y, value);
    While (y > x) {
        if (d <= 0) { /* Choose E */
            d += 2.0*x + 3.0;
        }
        else { /* Choose NE */
            d += 2.0*(x - y) + 5.0;
            y--;
        }
        x++;
        CirclePoints(x, y, value);
    } /* While */
} /* MidPointCircle */

```

The midpoint circle scan-conversion algorithm.

Problem: $d = 5.0/4.0 - \text{radius};$ $h = d - 1/4 \Rightarrow d = 1 - R$

```

Void MidPointCircle (      /* Assumes center of circle is at the origin
*/
    int radius,
    int value)          /* Integer arithmetic only */
{
    int x = 0;
    int y = radius;
    double d = 1 - radius; /* Initial value of d */
    CirclePoints(x, y, value);
    While (y > x) {
        if (d <= 0) {      /* Choose E */
            d += 2*x + 3;
        }
        else {             /* Choose NE */
            d += 2*(x - y) + 5;
            y--;
        }
        x++;
        CirclePoints(x, y, value);
    } /* While */
} /* MidPointCircle */

```

The integer midpoint circle scan-conversion algorithm.

Aliasing and anti-Aliasing

(غير مطالبين بالتفاصيل الرياضية لهذه الفكرة فقط الفكرة العامة النظرية)

أي كائن هندسي (خط، دائرة، مضلع) هو شكل مستمر (Continuous):
دقة لا نهائية (infinite resolution) لكن الشاشة:

▪ شبكة بكسلات

▪ دقة محدودة (Discrete)

عندما نحاول تمثيل:

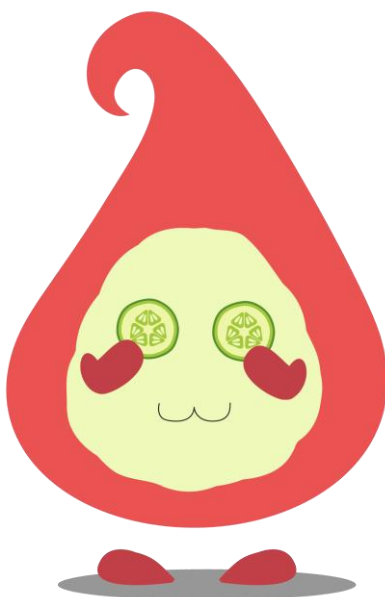
Continuous object → Discrete pixels

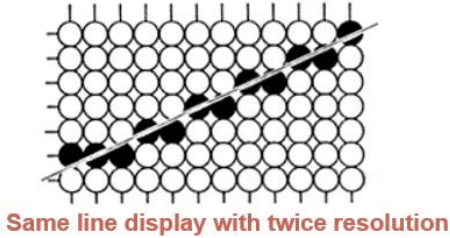
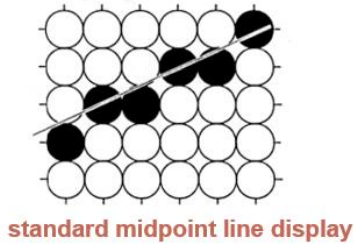
يظهر تشوه بصري اسمه: Aliasing ويكون:

▪ الخطوط تظهر متعرجة (jagged / stair-step)

▪ الحواف غير ناعمة

▪ واضح جداً مع الخطوط المائلة والمنحنيات



هل زيادة الدقة تحل المشكلة؟

هذه الصورة تقارن بين:

دقة عادية

دقة مضاعفة

نجد أن: التعرّج أقل لكنه لم يختفِ

ولا نستطيع زيادة الدقة دائماً لأن عدد البكسلات يزداد بشكل كبير، ويصبح الرسم أبطأ لأن الحسابات أصبحت أكثر، لذلك نحتاج إلى حل أفضل

Anti-aliasing (مكافحة التشوه) :

يُطلق على تطبيق التقنيات التي تقلل من التشوه اسم "مكافحة التشوه" وهي في جوهرها مشكلة متعلقة بأخذ العينات ف باستخدام هذه التقنية يتم إنتاج خط أكثر سلاسة عن طريق تحديد شدة وحدات البكسل النشطة وفقاً للنسبة المئوية للخط المثالي الذي يمر عبرها ففي السابق، كنا نحدد قيمة البكسل إما إلى أقصى شدة (Imax) أو إلى الصفر

لدينا مشكلة Aliasing أو Stair Case ، أي ظهور مستقيم بشكل متكرر.

بما أن الكمبيوتر لا يتعامل مع قيم مستمرة (يقوم بالتقطيع) فقطعنا الصورة إلى البكسل، و ذكرنا في مقرر الاتصالات و عندما تحدثنا عن تقطيع الصوت أنه إذا كان تردد التقطيع أعلى من ضعفي أعلى تردد للإشارة لا تضيع الإشارة. و يمكننا إسقاط هذه الفكرة على الصورة أيضاً حيث:

- تردد التقطيع في الصورة هو عدد البكسلات في الانتش.
- تردد الصورة يتحدد بعدد البكسلات لسوداء التي تجاور بكسلات بيضاء.
- لكيلا تضيع الإشارة (حل مشكلة Aliasing):
- إما نرفع تردد التقطيع: أي نزيد عدد البكسلات في الإنش.
- أو نخفض تردد الصورة وذلك بإضافة بكسلات رمادية.

■ إذا لدينا حلان:

1. زيادة عدد البكسلات: لكن هذا الحل مكلف مادياً و أيضاً و مكلف من جهة الذاكرة و السرعة.
2. إضافة بكسلات مادية: نقوم بإعطاء كل بكسل قيمة intensity عندما تكون صفر يظهر باللون الأبيض، و كلمات زادت تزداد درجة السواد، وهو الحل الذي سنستخدمه.

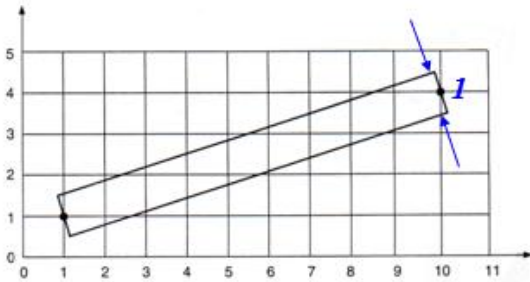
لدينا طريقتان لتطبيق هذا الحل:

1. Un-Weighted Area Sampling :

نعتبر:

كل بكسل = مربع وحدة (Unit Square)

الخط المثالي يمر داخل هذا المربع

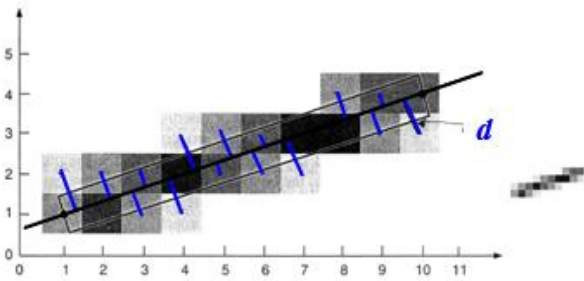


نسأل: كم نسبة مساحة هذا المربع التي يغطيها الخط؟

فبدلاً من أن نختار: pixel ON or OFF

نختار: pixel intensity $\propto$ area covered by line

أي أن شدة البكسل تتناسب طردياً مع المساحة التي يغطيها الخط.



ملاحظة هامة:

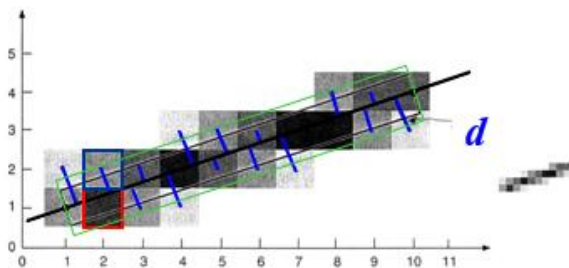
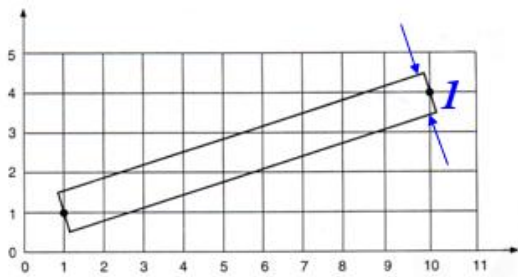
إذا أردنا رسم شكل وراء هذا المستقيم لن يكون المظهر واقعي (سيظهر البكسل الرمادي مثلاً و سيغطي على اللون الأسود للبكسل الذي يقع خلفه)، في التطبيق العملي نحل هذه المشكلة بأننا لا نغير الكثافة اللونية للبكسل و إنما نغير شفافيته.

مثال:

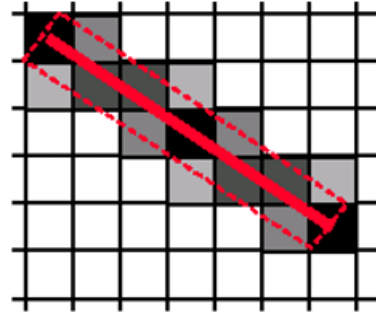
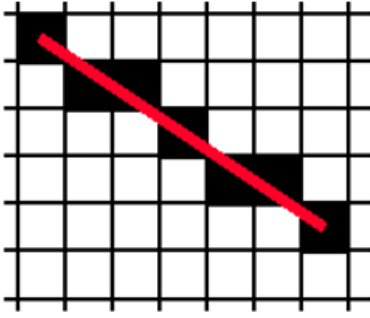
يساهم الخط المثالي في شدة إضاءة كل بكسل بمقدار يتناسب مع النسبة

(70 %) أسود بنسبة 2,1 المئوية لجزء البكسل الذي يغطيها، على البكسل

(تقريباً، وهكذا. 25) أسود بنسبة 2,2 تقريباً، والبكسل)



هذه الصورتان توضحان الفرق بين Aliasing and anti-Aliasing:



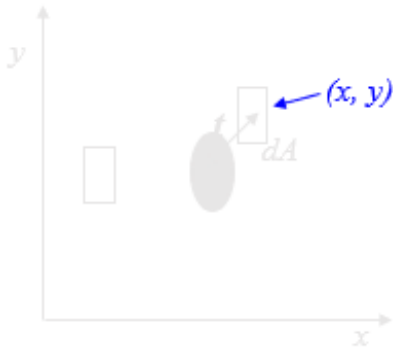
خصائص هذه الطريقة:

1. تساهم المساحات المتساوية بشدة متساوية، بغض النظر عن المسافة بين مركز البكسل والخط المحدد.
2. تنخفض شدة البكسل الذي يتقاطع مع قطعة الخط كلما زادت المسافة l بين مركز البكسل والخط المحدد. سيتم تظليل البكسلات الأقرب إلى المركز بشدة أعلى من تلك الأبعد.
3. لا يمكن للعنصر الأساسي أن يؤثر على شدة البكسل على الإطلاق إذا لم يتقاطع مع البكسل.

2. Weighted Area Sampling :

لا يؤثر كل جزء من الشكل على البكسل بنفس الدرجة حتى لو كان مساحتان متساويتان من الخط داخل البكسل فإن:

- الجزء الأقرب لمركز البكسل يؤثر أكثر
- الجزء الأبعد عن المركز يؤثر أقل



نعرف دالة وزن (فلتر): $W(x, y)$ تعتمد على المسافة t بين: مركز البكسل والنقطة (x, y) من الشكل داخل البكسل. كل عنصر مساحة صغير جداً: dA يساهم في شدة البكسل بحسب وزنه.

تخيل:

$W(x, y)$ سطح ثلاثي الأبعاد و المحوران x, y على مستوى الشاشة، والارتفاع فوق المستوى = الوزن كلما اقتربنا من مركز البكسل:

- الارتفاع أكبر
- التأثير أكبر



تعريف المساحة المغطاة داخل البكسل:

نعرف:

$$\Delta A = \int dA$$

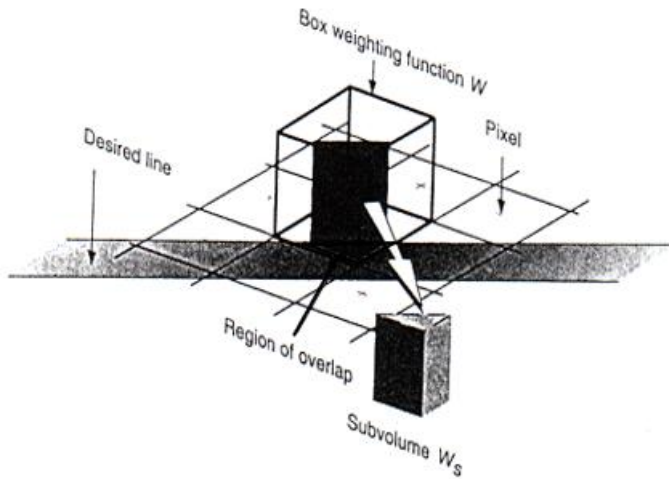
وهي: المساحة من البكسل التي يغطيها الخط أو المنحني.

تعريف sob-volume لكل بكسل:

$$W_S = \int W(x, y) dA$$

حيث: $1 \geq W_S \geq 0$ تعريف شدة إضاءة البكسل:

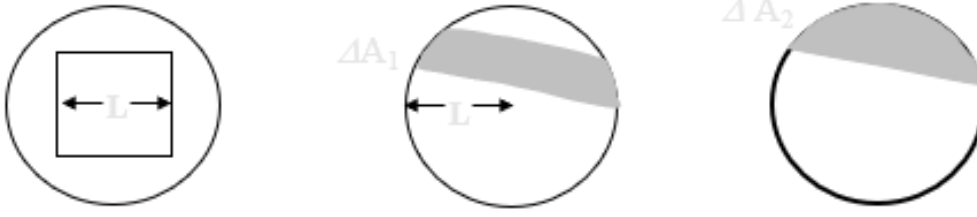
$$dI = W_S I_{max}$$

حيث: I_{max} هي الشدة القصوى (لون الخط الكامل)في الحالة البسيطة (بدون Anti-Aliasing): $W(x, y) = constant$ يعني:

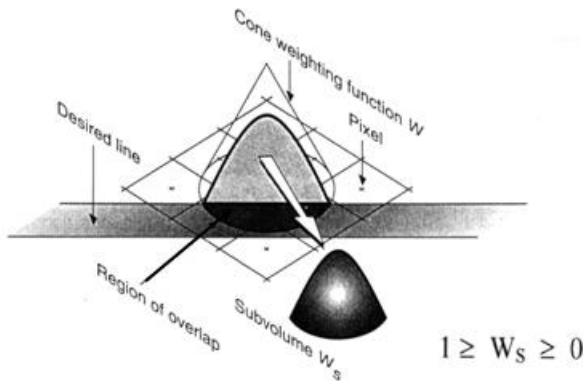
- كل النقاط داخل البكسل لها نفس التأثير
- لا نهتم بالمسافة عن المركز
- أي أن: $\Delta A_1 = \Delta A_2$
- إذن: $dI_1 = dI_2 \Rightarrow W_{s1} = W_{s2}$
- وهذا يؤدي إلى:
- حواف حادة
- ظهور التكسر (Aliasing)

طريقة أكثر فعالية:

ربط مساحة أكبر قليلاً بكل بكسل وتطبيق دالة وزن مختلفة، $W(x, y)$.
لتبسيط الأمر، يتم ربط كل بكسل بقاعدة دائرية نصف قطرها يساوي طول ضلع شبكة البكسل.



- تساهم المنطقة ΔA الأقرب إلى مركز البكسل بشكل أكبر في شدة الإضاءة من المنطقة الأبعد.
- قد تساهم قطعة صغيرة واحدة من المنطقة في شدة إضاءة عدة بكسلات مختلفة

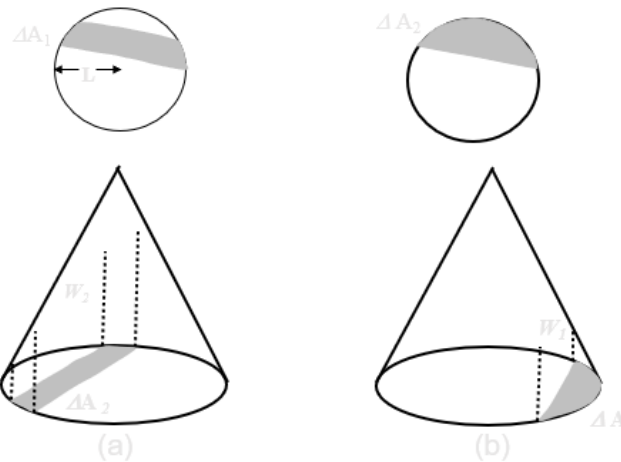


خصائص الدالة $W(x,y)$:

- تكون قيمتها قصوى عند مركز البكسل
- تتناقص خطياً مع زيادة المسافة
- على سبيل المثال، يمكن اختيار الدالة $W(x, y)$ لتكون دالة وزن مخروطية
- يتطابق مركز المخروط الدائري مع مركز البكسل
- تُعطى قيمة $W(x, y)$ بارتفاع المخروط
- تكون قيمة $W(x, y)$ لمنطقة معينة أكبر كلما كانت أقرب إلى المركز.

مثال لإثبات أن طريقة Weighted Area Sampling فعالة:

- لنفرض أن لدينا مساحتين متساويتين من "كائن" ما (مثلاً خط أو مضلع)، ولنسمهما ΔA_1 و ΔA_2 .
- المساحة ΔA_2 قريبة جداً من المركز.
 - المساحة ΔA_1 بعيدة عند الحافة.



النتيجة:

بما أن الوزن عند المنطقة الثانية $W(x_2, y_2)$ أكبر من الوزن عند المنطقة الأولى $W(x_1, y_1)$ ، فإن مساهمة المنطقة الثانية في شدة إضاءة البكسل ستكون أكبر بكثير

المعادلة الرياضية: التغير في شدة الإضاءة dI يحسب كالتالي:

$$dI = W_S \cdot I_{max} = I_{max} \int W(x, y) dA$$

حيث إن:

- W_S هو الوزن المحسوب للمنطقة.
- I_{max} هو أقصى شدة إضاءة ممكنة.
- التكامل $\int W(x, y) dA$ يمثل هندسياً "الحجم" الموجود تحت المخروط وفوق مساحة الجسم المارة بالبكسل.

الخلاصة في هذه الطريقة:

نحن لا نكتفي بحساب "كم" يغطي الجسم من البكسل، بل نهتم بـ "أين" يغطي الجسم هذا البكسل. المساحات المركزية هي التي تقرر اللون النهائي للبكسل بشكل أساسي.

The End



Success begins the moment you decide not to quit